In [ ]:
```
# Run this cell first.
# In the browser (JupyterLite) it installs the extra packages into this t
# in Colab or Anaconda it installs them with pip (already installed = no
%pip install ipywidgets
```

Week 1 Tutorial - Introduction to OR

Before completing this tutorial you should complete the notebook **Using NumPy for Linear Algebra** (`2 Lecture Demos/1 Vectors and matrices in NumPy.ipynb` in this week's folder).

You also might want to have it open to refer to while trying to complete this tutorial.

The following exercises are aimed at graphing inequalities and visualising systems of equations using Python, there are many programs (software) which can do this, however we will focus on Python throughout the course for consistency.

A good understanding of vectors and matrices will really prove beneficial when studying topics in Operational Research such as Linear Programming, Integer Programming and Networks to name a few.

P.S. If you want to learn some LaTeX, you can double-click on the cells to see the LaTeX code.

How to use this notebook: run the first cell once (it installs `ipywidgets` for the 3D slider in Exercise 8), then work down the page, pressing Shift+Enter to run each cell. Exercises 1, 3a–b and 6 are pen-and-paper; Exercises 5–6 use desmos.com. **By the end you should have:** three determinants, the solution of the Exercise 3 system and a plot of its two lines crossing, a shaded feasible region, and the solution of the 3 × 3 system in Exercise 7 with a 3D picture of three planes meeting at one point. Check your work against `4 Tutorial Answers` afterwards.

Exercise 1

Find the determinant of the following matrices by hand:

a.

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

b.

$$\begin{pmatrix} 1 & -2 \\ 3 & 4 \end{pmatrix}$$

c.

$$\begin{pmatrix} 2 & 6 \\ 4 & 12 \end{pmatrix}$$

What can you say about a system of equations that has each of these matrices?

Exercise 2

Use Python to find the determinants and check your answers to Exercise 1.

I have provided a runnable cell for you. Just click in the cell and press Shift+Enter to run the code.

We will be using NumPy which is a library that supports vectors/matrices and their operations. In NumPy they are called arrays.

In [5]:
```
import numpy as np

# Place your answer here for Exercise 2
print("Write your code here for Exercise 2!")
```

```
Write your code here for Exercise 2!
```

Exercise 3

a) Solve the following system of equations by hand using elimination or substitution.

$$3x + y = 11 \quad (1)$$
$$x + 4y = 11 \quad (2)$$

b) Solve using matrix form

c) Use Python to solve this system.

In [6]:
```
import numpy as np

# Place your answer here for Exercise 3
print("Write your code here for Exercise 3!")
```

```
Write your code here for Exercise 3!
```

Exercise 4

Produce a plot of your solution to Exercise 3 using Python.

Amend the code below. As given, it solves an example system, $x + 2y = 1$ and $3x + 4y = 3$ (solution $(1, 0)$). It rewrites each equation $a_{i1}x + a_{i2}y = b_i$ as $y = (b_i - a_{i1}x)/a_{i2}$, draws both lines around the solution and marks the solution

with a black dot. Replace `A` and `b` with the Exercise 3 system; the lines should then cross at your answer.

In [7]:
```
import matplotlib.pyplot as plt

# Here it is in python!

# This is a square matrix with 2 rows and 2 columns (2,2).
A = np.array([[1,2],
              [3,4]
              ])

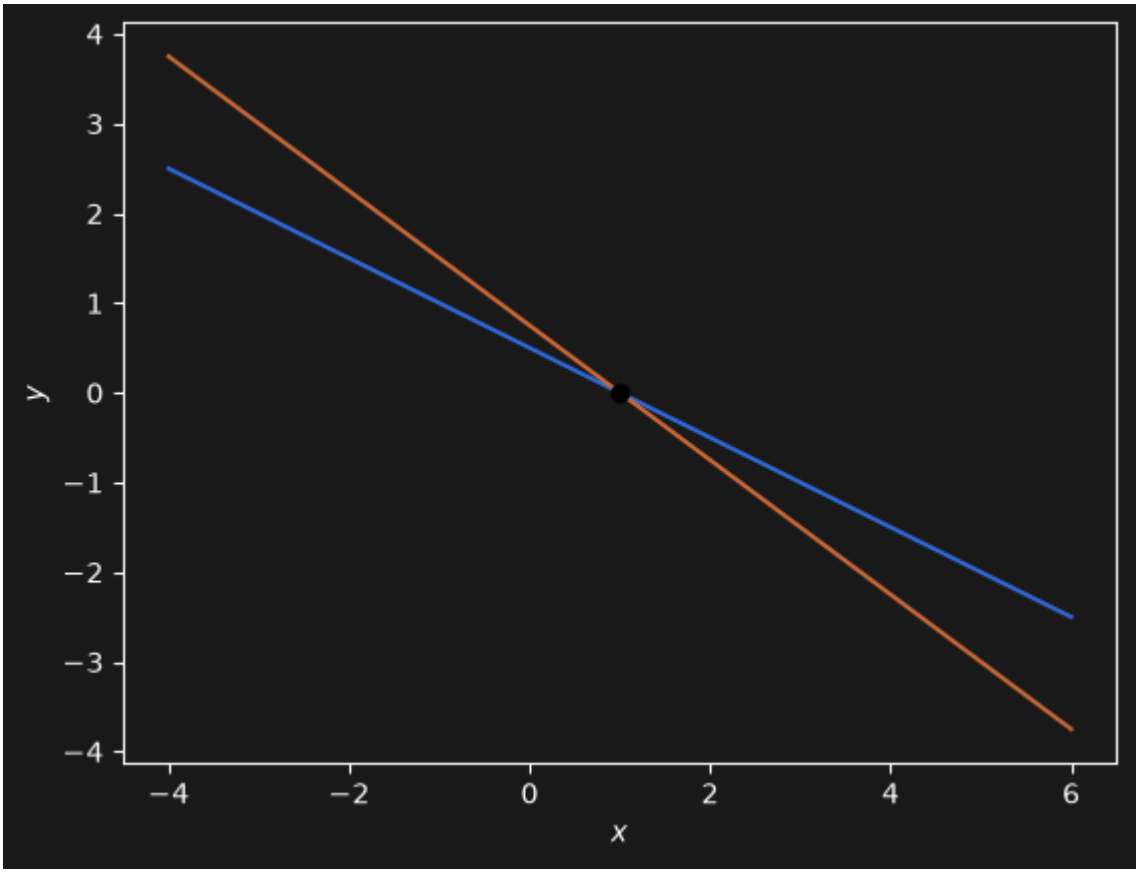
b = np.array([1,3])

# We can solve a system of linear equations using the matrix inverse, np.
x = np.dot(np.linalg.inv(A), b)

# 100 x values from 5 below to 5 above the solution
X = np.linspace(x[0] - 5, x[0] + 5, num=100)
# equation 1 solved for y: y = (b1 - a11*x) / a12
Y1 = (b[0] - A[0,0]*X)/A[0,1]
# equation 2 solved for y
Y2 = (b[1] - A[1,0]*X)/A[1,1]

plt.plot(X,Y1)
plt.plot(X,Y2)
plt.plot(x[0], x[1], "ko")   # the solution: 'k' = black, 'o' = dot
plt.xlabel("$x$")
plt.ylabel("$y$")

plt.show()
```



Exercise 5

Produce a plot of your solution to Exercise 3 on desmos.com

Exercise 6

Graph the following system of inequalities by hand. Shade the feasible region.

$$x + y \leq 2 \tag{3}$$
$$x + 3y \leq 3 \tag{4}$$
$$x \geq 0 \tag{5}$$
$$y \geq 0 \tag{6}$$

Confirm this by plotting this on desmos.com

Exercise 7

Solve the following system using Python.

$$2x + 3y - 2z = 3 \tag{7}$$
$$x - 2y + 4z = 0 \tag{8}$$
$$2x - 3y + z = 1 \tag{9}$$

You will need to work out $\mathbf{A}$ and $\mathbf{b}$.

$$\mathbf{A}\mathbf{x} = \mathbf{b} \tag{10}$$
$$\begin{pmatrix} * & * & * \\ * & * & * \\ * & * & * \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} * \\ * \\ * \end{pmatrix} \tag{11}$$

In [8]:
```
import numpy as np

# Place your answer here for Exercise 7
print("Write your code here for Exercise 7!")
```

```
Write your code here for Exercise 7!
```

Exercise 8

Visualise your solution to Exercise 7 by amending the following code.

As given, the code uses an example 3 × 3 system. It solves it, rewrites each equation as $z = (b_i - a_{i1}x - a_{i2}y)/a_{i3}$ on a 20 × 20 grid of (x, y) points from −10 to 10, and draws the three planes. Replace `A` and `b` with the Exercise 7

system. `ax.view_init(0, 0)` looks along the x-axis; change the two angles (elevation, azimuth, in degrees) to rotate.

In [9]:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

A = np.array([
              [ 2,  3,  1],
              [-1,  6,  1],
              [-1, -3,  1]
              ])

b = np.array([4, 2, -4])

# We can solve a system of linear equations using the matrix inverse, np.
x = np.dot(np.linalg.inv(A), b)

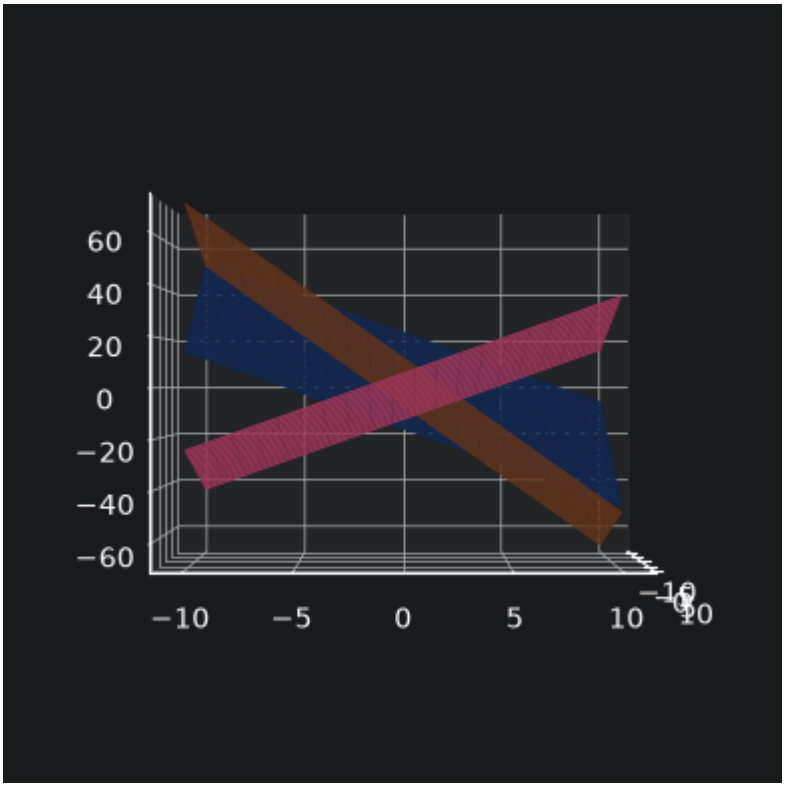
# 20 x 20 grid of (x, y) points
X = np.linspace(-10, 10, num=20)
Y = np.linspace(-10, 10, num=20)
xv, yv = np.meshgrid(X, Y)

# each equation solved for z: z = (b_i - a_i1*x - a_i2*y) / a_i3
z1 = (b[0] - A[0,0]*xv - A[0,1]*yv)/A[0,2];
z2 = (b[1] - A[1,0]*xv - A[1,1]*yv)/A[1,2];
z3 = (b[2] - A[2,0]*xv - A[2,1]*yv)/A[2,2];


fig, ax = plt.subplots(subplot_kw={"projection": "3d"})

# https://matplotlib.org/stable/api/_as_gen/mpl_toolkits.mplot3d.axes3d.A
ax.view_init(0, 0) # use this to manually rotate your plot
ax.plot_surface(xv,yv,z1)
ax.plot_surface(xv,yv,z2)
ax.plot_surface(xv,yv,z3)

plt.show()
```



I have also provided an interactive version of the plot. Drag `elev` (up/down) and `azim` (round) to rotate it. The planes are see-through and the black dot is the solution. To show Exercise 7, change `A` and `b` inside the function.

In [10]:
```
from ipywidgets import interact, fixed
# Importing mplot3d toolset
from mpl_toolkits.mplot3d import Axes3D



@interact(elev=(-90, 90), azim=(-180, 180))
def plot_3D(elev=30, azim=30):
  A = np.array([
              [ 2,  3,  1],
              [-1,  6,  1],
              [-1, -3,  1]
              ])

  b = np.array([4, 2, -4])
  # We can solve a system of linear equations using the matrix inverse, n
  x = np.dot(np.linalg.inv(A), b)

  # Grid of (x, y) points on which the three planes are drawn
  X = np.linspace(-10, 10, num=20)
  Y = np.linspace(-10, 10, num=20)
  xv, yv = np.meshgrid(X, Y)

  # each equation solved for z: z = (b_i - a_i1*x - a_i2*y) / a_i3
  z1 = (b[0] - A[0,0]*xv - A[0,1]*yv)/A[0,2];
  z2 = (b[1] - A[1,0]*xv - A[1,1]*yv)/A[1,2];
  z3 = (b[2] - A[2,0]*xv - A[2,1]*yv)/A[2,2];


  fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
```

```
  # https://matplotlib.org/stable/api/_as_gen/mpl_toolkits.mplot3d.axes3d
  ax.view_init(elev, azim) # use this to manually rotate your plot
  ax.scatter(x[0], x[1], x[2], c='k', s=20)
  ax.plot_surface(xv,yv,z1, alpha=0.3)
  ax.plot_surface(xv,yv,z2, alpha=0.3)
  ax.plot_surface(xv,yv,z3, alpha=0.3)


  plt.show()
```

```
interactive(children=(IntSlider(value=30, description='elev', max=90, min=
-90), IntSlider(value=30, descriptio…
```